

Chedam — Spec v0.2: Change Requests

Oct 2, 2026 · @sreya brahma teja

This document adds nine change requests to the v0.1 specification. Where they conflict, v0.2 wins; everything else in v0.1 stands.

1. Summary of changes

All nine requests are feasible. Two need things the v0.1 design avoided: delivery-platform orders need the hub to go online (outbound only), and camera cash counting needs a stronger device than the Pi Zero 2 W.

#	Request	Decision	Changes v0.1 section
1	Owner access to all devices	Owner can log in on any device, see and manage every device, and send to all devices	3 Users and roles
2	Access editable by owner and manager	Per-person permission overrides on top of roles; managers can edit everyone except owners	3 Users and roles
3	Hold sales during sync to avoid duplicates and negatives	Hub processes stock-changing actions one at a time; offline tills keep selling and conflicts are flagged	4 Data flow, 7 Offline behaviour
4	Estimate till cash at open and close	Camera photo of the drawer gives an estimate by denomination; cashier confirms	11.1 Till balances
5	Add stock from a phone by barcode	Scan to find the product or open a new-product form with the barcode filled	11.2 Inventory
6	Quick bulk add	Scan-row grid: barcode, price per unit, quantity, add next row	11.2 Inventory

#	Request	Decision	Changes v0.1 section
7	Packs and single units	One product, several selling units linked by a conversion; breaking a pack asks for confirmation	8 Data model, 11.1, 11.2
8	Uber Eats, DoorDash, SkipTheDishes orders	Automatic when internet is available, quick manual entry as fallback; stock updates on accept and complete	New module; 2 Scope
9	Vendor bill photo to inventory	Photograph the bill, extract lines, match to products, review, then receive into stock	11.2 Inventory, 11.5 Vendors

2. Roles: owner reach and editable access

The owner can reach every device, and access is set per person, not only per role, because duties in a small business change often.

Owner access to all devices

- **Log in on any device:** the owner's password or PIN works on every till, PC and phone, even if that device is assigned to someone else.
- **Device manager screen** (owner; managers view only):
 - list of every device: name, type, online/offline, last seen, who is logged in, app version, battery (phones)
 - actions: lock screen, log out user, rename, assign to a person or till, revoke (lost or stolen), approve a new device
- **Send to all devices:** one ping or announcement to every device, or to a group (all tills, all phones, back office). Urgent ones show full screen until acknowledged, and the owner sees who has acknowledged.
- Every owner action on another device is written to the event log.

Editable access

- Roles stay as templates (Owner, Manager, Cashier, Staff, Accountant).
- Each person can have **overrides**: switch individual permissions on or off without changing their role (e.g. a cashier who also receives stock today).

- **Temporary access:** an override can have an end date ("Priya can approve refunds until Sunday"); it switches off automatically.
- **Who can edit what:**

Editor	Can change	Cannot change
Owner	Everyone's role and overrides, including managers; role templates	Remove the last owner
Manager	Roles and overrides of cashiers, staff, accountants, and other managers' non-financial permissions	Owners; pay rates and HR visibility; giving anyone more access than the manager has

- Access changes take effect immediately on all devices and appear in the event log with who changed what.
- A "**Who can do this?**" view lists, for any permission, every person who currently has it.

3. Sync and concurrency: no duplicates, no negative stock

The hub, not the tills, does the holding: every action that changes stock or money waits in a single queue on the hub and is processed one at a time. Each takes a few milliseconds, so cashiers never see a pause, and two tills can never sell the same last unit.

How it works (online)

1. A till finishes a sale and sends it to the hub with a unique sale ID created on the till.
2. The hub puts it in the stock-and-money queue. Only one item from the queue runs at a time.
3. For that sale the hub, inside one database transaction: checks the sale ID is new (duplicate → returns the original result, nothing is written twice), checks available stock for every line, deducts stock, saves the sale and payments, writes the event log.
4. If any line would go below zero, the whole sale is rejected **before** payment is completed and the till shows: "Only 2 left of Mango Pulp 850 g — adjust quantity".
5. The hub broadcasts the new stock levels; every till's screen updates within about a second.

What else uses the same queue

Sales, returns, stock receiving, stock adjustments, pack breaking, delivery-platform orders, till open/close, cash drops. Messages, pings and reading data do not wait in it.

Reserving during checkout (optional setting)

For very low-stock items (setting, default: 3 or fewer left), adding the item to a cart places a soft hold for 5 minutes, so another till sees "1 held at Till 2". The hold drops when the sale completes, the line is removed, or time runs out.

When a till cannot reach the hub (offline)

Decision: **keep selling, flag conflicts.**

- The till sells from its cached stock and queues sales locally (each with its unique ID).
- Offline tills show an amber "Offline — stock may be out of date" bar.
- On reconnect, queued sales go through the same hub queue in time order. Duplicates are ignored by ID.
- If a queued sale would make stock negative, it is still accepted (the customer already left with the goods), stock is set to the true negative figure, and an **Urgent** task is raised: "Oversold 2 × Mango Pulp while Till 2 was offline — count and adjust".
- Negative stock is never hidden; reports show it in red until counted.

4. Till open and close: camera cash estimate

The cashier photographs the open drawer, the app estimates the cash by denomination, and the cashier corrects any count before saving. The estimate speeds counting; the confirmed count is what gets saved.

Flow

1. Open or close till → tap **Estimate with camera**.
2. A guide frame shows how to hold the phone/tablet: drawer fully open, top-down, good light. Notes fanned in their slots so edges show.
3. One photo of the note and coin compartments (optional second photo of notes spread on the counter for better accuracy).
4. The app shows a count per denomination with a confidence colour (green sure, amber check, red guess) and the estimated total.
5. Cashier adjusts any row with + / – and taps **Confirm count**. Expected vs counted difference is shown as in v0.1.
6. The photo is stored with the till record for 90 days (audit), visible to managers and the owner.

What it can and cannot do

- **Coins** (5¢ to \$2) are recognised well from size and colour when spread in compartments; stacked coins are estimated from visible stack height and flagged amber.
- **Notes** (\$5–\$100) are recognised by colour, but counting notes stacked flat in a slot is unreliable. Fanned notes or a counter spread are needed for good accuracy.
- **Target:** estimate within about 5% of true total for a well-taken photo; the cashier's confirmation is always required.

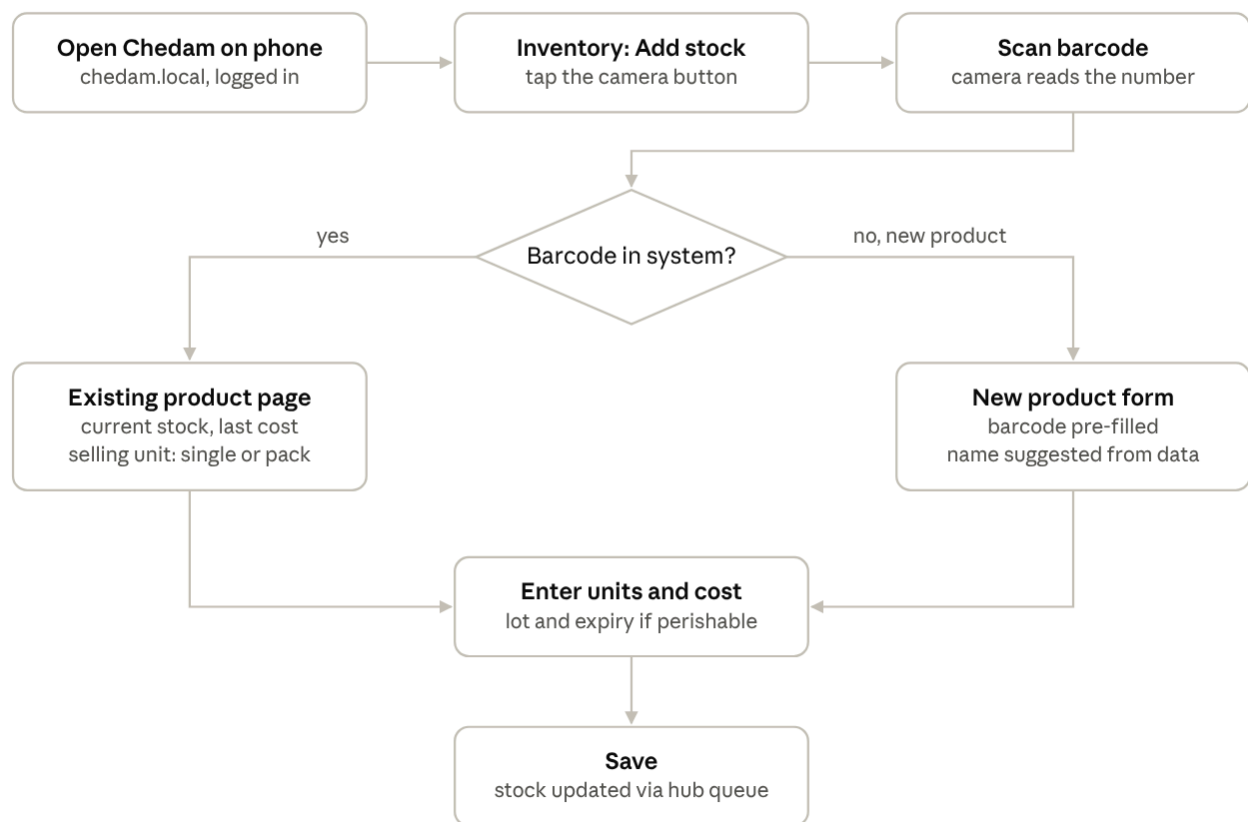
Where it runs

- Image recognition is too heavy for the Pi Zero 2 W (512 MB).
- **Standard tier:** the feature is hidden; cashiers count by denomination (v0.1).
- **Premium tier (stronger hub or AI device):** a small object-detection model trained on Canadian coins and notes runs on the hub or in the browser of a capable tablet.
- **Training data:** we photograph real drawers and label them ourselves (a few hundred photos per denomination); the model is bundled with the premium package.

5. Add stock by phone

Adding stock takes three taps and a scan: open Inventory on the phone, scan the barcode, enter the quantity. A known barcode opens that product; an unknown one opens a new-product form with the barcode already filled.

Every scan lands on a product page, existing or new



add stock by phone · scan, match, save

Both branches end on the same quick entry and save through the hub queue, so stock on every till updates within about a second.

Details

- **Camera:** the phone's browser reads EAN-13, UPC-A, EAN-8, Code 128 and QR codes. Phones need the hub's trusted certificate (set up during device pairing) because browsers only allow the camera on secure connections.
- **Can't read it?** "Type the code" is always on the camera screen; a torch button helps in coolers.
- **Existing product page:** shows a product photo, stock by selling unit (sealed packs and loose, Section 7), last cost and vendor. Fields: units to add, cost per unit (pre-filled with last cost), lot and expiry (perishables), storage area. One tap **Save**, then **Scan next**.
- **Same barcode on a pack and a single:** the app asks "Adding packs or singles?" before the quantity.
- **New product form:** barcode filled; name, brand, size, ingredients and allergens suggested from the bundled product data (v0.1 Section 13) when the barcode is found there; category, HS code, shelf life and storage suggested from the name. Required:

name, selling unit, price. Everything else can be finished later; a task reminds the user.

- **Permissions:** staff with "Receive stock" can add stock; creating new products needs "Edit products" or goes to a manager as a draft for approval (setting).
- **Offline:** if the phone loses the hub, the entry is kept on the phone and sent when it reconnects.

6. Quick bulk add

Quick bulk add is a scan-as-you-go grid for receiving many products at once: one row per product, filled by the camera, saved in one tap.

Screen

Column	Filled by	Notes
Scan	Tap → camera reads barcode	Or type/paste the code
Product	Auto from barcode	Green = known product; amber = new (name required); tap to open the full form
Unit	Auto (each / pack of 12 / kg)	Selling unit chosen for this barcode (Section 7)
Price per unit (cost)	Last cost pre-filled	Editable; changes vs last cost shown as ▲/▼ %
Units to add	Default 1	+/- buttons; scanning the same barcode again adds 1 to that row
Lot / expiry	Optional	Shown for perishables; required if the product is marked perishable
×	—	Remove row

Under the last row: + **Add product** (opens camera for the next row). A footer shows rows, total units and total cost.

Rules

- **Continuous scan mode:** the camera stays open and adds a row (or +1) per scan, with a beep and vibration.
- Rows for new products are saved as drafts with name + barcode + cost; they appear as a task "Complete 3 new products" so the details can be finished later.

- Optional fields at the top apply to all rows: vendor, purchase order, storage area, received date.
- **Save** sends all rows through the hub queue in one transaction (all or nothing) and shows a receipt summary.
- Unsaved grids are kept on the phone, so a dropped connection or closed app loses nothing.
- If a vendor and PO are chosen, quantities are checked against the PO and differences flagged (short, over, unexpected item).

7. Packs and single units

A product has one stock count kept in its smallest unit, and any number of **selling units** (single, 6-pack, case of 24) linked to it by a conversion. Packs sell as packs; when a single is sold and no loose singles are left, the app asks to open a pack and moves the stock with the user's confirmation.

Example: Mango Lassi 200 ml

Selling unit	Barcode	Contains	Price	Stock shown
Single bottle	8901234500011	1	\$2.49	7 loose
6-pack	8901234500066	6 singles	\$12.99	4 sealed packs
Case	8901234500240	4 × 6-pack (24)	\$45.00 (wholesale)	0 sealed cases

Total stock = $7 + 4 \times 6 = 31$ bottles. Cost and margin are calculated per single, so each unit shows its own margin.

Data model change

- New table `selling_units`: product, name, barcode(s), contains (qty of base unit or of another selling unit), price, sell at POS (yes/no), tax class override.
- `stock_levels` stores **sealed** counts per selling unit plus **loose** base units.
- `stock_movements` gains type `pack_break` (and `pack_make` for the reverse, e.g. assembling gift boxes).

Opening a pack

1. A single is scanned at the till while loose stock is 0 but sealed 6-packs exist.
2. The till shows: "No loose Mango Lassi left. Open a 6-pack? (4 sealed packs)" with **Open 1 pack / Cancel**.

3. On confirm: 1 pack → 6 loose, then the single sells (5 loose left). Recorded as `pack_break` by that cashier.
4. Packs can also be opened deliberately from Inventory (e.g. stocking the single-bottle shelf), choosing how many.
5. If a sealed pack is opened and some units are damaged, the user records them as damaged in the same screen.

Rules

- Selling units can nest (case → 6-pack → single).
- Reorder points can be set on either sealed or total stock.
- Price check: the app warns if a pack costs more than buying its singles.
- Imports (v0.1 Section 12) detect pack sizes in names ("6x200ml", "Case/24") and suggest selling units.

8. Delivery platform orders (Uber Eats, DoorDash, SkipTheDishes)

Decision: **automatic when internet is available, quick manual entry as the fallback.**

Manual entry ships first, because automatic connection to these platforms requires partner approval that Chedam does not have yet.

What the platforms require (checked October 2026)

Platform	How POS systems get orders	Access
Uber Eats	Marketplace API; orders pushed to the POS by webhook	Written approval from Uber, NDA and API licence; POS integrations also need the merchant's Uber Eats partner manager to confirm. Typical build 4–8 weeks. (Uber)
DoorDash	Marketplace API; orders by webhook; menu sync	Not generally available; apply via the developer portal; certification with screen recordings; must keep order failure and merchant cancel rates under 1%. (DoorDash)
SkipTheDishes	Through integration partners/aggregators (e.g. Cuboh, Deliverect)	No public self-serve API found; aggregators charge a monthly fee. (Cuboh)

All three push orders to a public internet address, so automatic mode needs the hub to be reachable from the internet, not just online.

Mode A: manual entry (ships in v1, works offline)

- **Delivery orders** screen on the till: pick platform (Uber Eats, DoorDash, Skip, other), enter the platform order number and customer first name.
- Add items by tapping the same product grid as the till, or by scanning.
- **Platform menus:** each product can have a platform name and price (platforms often charge more). Saved once, reused.
- **Snap the ticket (premium):** photograph the platform tablet's order screen or printed ticket; the app reads items with the same reader as vendor bills (Section 9) and fills the order for confirmation.
- **Statuses:** **Accepted** (stock reserved) → **Preparing** → **Picked up** (stock deducted, sale recorded under that platform) or **Cancelled** (reservation released).
- Payment method = the platform; amounts reconcile later against the platform's payout statement (CSV import in Reconciliation).

Mode B: automatic (later phase, needs internet)

- The hub gets an internet connection and a secure tunnel so platforms can reach it (e.g. Cloudflare Tunnel, free tier) — only the order endpoint is exposed, nothing else.
- **Route 1 — aggregator:** the business subscribes to an aggregator (Cuboh, Deliverect, or similar) and Chedam connects to its single API. Fastest, but adds a monthly cost to the business.
- **Route 2 — direct:** Chedam applies to Uber and DoorDash as a POS partner and gets certified. No fee to the business, but a business milestone for Chedam (company, agreements, certification), not only coding.
- Incoming orders appear on every till with a sound, follow the same statuses as Mode A, and update stock through the hub queue (Section 3).
- **Menu sync:** stock reaching 0 can mark the item unavailable on the platform automatically.
- If the internet drops, platforms fall back to their own tablets and staff switch to Mode A; nothing is lost.

Inventory rule (both modes)

Accepted = reserve, picked up = deduct, cancelled = release. Payouts and platform commission are recorded so margins show the true profit per delivery order.

9. Vendor bill photo to inventory

The user photographs or uploads a vendor's delivery bill; Chedam reads the lines, matches them to products, and opens a pre-filled receiving grid (the Quick bulk add screen) for review before anything enters stock.

Flow

1. Inventory → **Receive from bill** → take photo(s) or upload a PDF/image (multi-page supported).
2. **Read:** text recognition extracts vendor name, bill number, date, and each line: item description, vendor code, quantity, unit, unit price, line total, tax.
3. **Match vendor:** to an existing vendor by name, phone or GST number; otherwise propose a new vendor.
4. **Match lines to products,** in this order: vendor item code saved from earlier bills → barcode on the bill → similar description (fuzzy match) → unmatched.
5. **Review grid:** each line shows the bill text beside the matched product, quantity, selling unit (case, pack, single — Section 7), cost, and a confidence colour. Unmatched lines get "Choose product" or "Create new".
6. **Checks:** sum of lines vs bill total; tax vs expected rate; cost change vs last cost (e.g. ▲ 12%); quantities vs the open purchase order.
7. **Confirm:** stock is received through the hub queue, costs and price history update, the bill is saved as a payable with the image attached.
8. **Learn:** every confirmed match is remembered (vendor code → product, "CS/24" → case of 24), so the next bill from the same vendor fills almost entirely by itself.

Where the reading runs

Tier	Engine	Notes
Standard	Tesseract.js (free, open source) in the phone or back-office browser	Works offline; good on printed bills; user fixes more lines
Premium	Stronger OCR + layout model (e.g. PaddleOCR) on the premium hub	Better on photos, columns, faded thermal paper

The Pi Zero 2 W never does the reading; it only receives the confirmed result.

Limits

- Handwritten bills read poorly; they go to the grid mostly blank for manual entry.

- Accuracy depends on the photo: flat, well lit, whole page in frame. The camera screen shows a guide frame and warns about blur.
- Nothing is saved to stock without the user tapping Confirm.

10. Hardware, software and cost impact

Most changes are software only and stay \$0. Camera cash counting and better bill reading move to the premium tier; automatic delivery orders bring an internet dependency and, by the aggregator route, a monthly fee paid by the business.

Change	New components	Runs on	Cost to business
Owner device access, editable access	Device manager screen, permission overrides	Hub (Pi Zero OK)	\$0
Hub queue, soft holds	Serialized write queue in PocketBase hooks	Hub (Pi Zero OK)	\$0
Phone barcode scan	Browser BarcodeDetector + ZXing fallback; needs trusted HTTPS on phones	Phone	\$0
Quick bulk add	Grid screen, local draft storage	Phone / PC	\$0
Packs and singles	<code>selling_units</code> table, pack-break movement	Hub	\$0
Delivery orders — manual	Delivery order screen, platform menus	Till	\$0
Delivery orders — automatic	Internet, Cloudflare Tunnel (free), platform/aggregator connector	Hub	Internet plan; aggregator fee if that route is used
Vendor bill reading — standard	Tesseract.js	Phone / PC browser	\$0
Vendor bill reading — premium	PaddleOCR + layout model	Premium hub	Premium package
Camera cash estimate	Coin/note detection model (we train it)	Premium hub or capable tablet	Premium package

Pi Zero 2 W check

Everything marked "Hub (Pi Zero OK)" adds little memory. Image work (bill reading, cash photos) is kept off the Zero by design. The tunnel for Mode B uses roughly 20–40 MB of RAM, so on the Zero it should be tested under load before relying on it.

11. Updated phase placement

The new features slot into the existing six phases; two items move to a new premium track that runs after Phase 5.

Phase (v0.1)	Added by v0.2
0 Foundation	Permission overrides and temporary access; device manager; owner log-in anywhere; hub stock-and-money queue; trusted HTTPS on phones (needed for camera)
1 Sell	Packs and singles with pack-break prompt; soft holds; offline conflict tasks; phone barcode scan to product page / new product; quick bulk add
2 Communicate	Send to all devices / groups with acknowledgement tracking
3 Buy and spend	Receive from vendor bill (standard OCR); vendor code learning; delivery orders manual mode (Mode A)
4 People and money	Platform payout reconciliation (CSV import)
5 Switch and enrich	Import detection of pack sizes
Premium track	Camera cash estimate; premium bill reading; snap-the-ticket for delivery orders
Partner track (business milestone)	Delivery orders automatic mode (Mode B): aggregator connector first, direct Uber/DoorDash certification later

12. Risks and open questions

Risks

Risk	Impact	Mitigation
Platform API access refused or slow	Automatic delivery orders delayed	Manual mode ships first; aggregator route as a bridge

Risk	Impact	Mitigation
Exposing the hub to the internet	Security	Tunnel exposes only the order endpoint; signed requests verified; everything else stays LAN-only
Phone camera blocked without trusted HTTPS	Barcode scan fails on phones	Certificate install built into device pairing; test iOS and Android in Phase 0
Cash photo accuracy on stacked notes	Wrong estimates	Cashier always confirms; fanned-note guide; premium only
Bill reading on thermal or crumpled paper	More manual fixes	Learning from confirmed matches; premium OCR
Pack conversions set wrong	Stock counts drift	Conversion shown on every pack screen; count report by sealed/loose
Managers over-granting access	Fraud risk	Managers cannot grant beyond their own access; all changes logged; owner "Who can do this?" view

Open questions

- ☐ Delivery: does the pilot business already use an aggregator (Cuboh, Deliverect, Otter)? If yes, Mode B can connect to it first.
- ☐ Delivery: does the pilot sell on all three platforms, or one first?
- ☐ Packs: do pack and single have different barcodes on your products, or the same barcode? (Same barcode needs a "pack or single?" prompt at scan.)
- ☐ Bills: which vendors' bills should we collect as test samples (aim for 10–20 photos)?
- ☐ Cash photo: is a tablet at the till acceptable, or must it work on any phone?
- ☐ Soft holds: is 3 units / 5 minutes the right default for your shop?